

Analyse d'un réseau de trafic modélisé par graphe

Joslyn Samuel

Mai 2026

Table des matières

1	Introduction	2
2	Modèle Erdős–Rényi	2
2.1	Choix de la probabilité	2
3	Génération du graphe	2
3.1	Algorithme	2
3.2	Code Python	3
4	Correction de la connexité	3
4.1	Algorithme	3
4.2	Code Python	3
5	Analyse du réseau	4
5.1	Densité	4
6	Classification des sommets	4
6.1	Code Python	4
7	Interprétation (fraude)	4
8	Conclusion	5

1 Introduction

Les réseaux complexes apparaissent dans de nombreux domaines : réseaux sociaux, fraude, transport ou encore communication.

Dans ce projet, nous modélisons un réseau d'interactions à l'aide d'un graphe aléatoire $G(n, p)$ afin d'analyser sa structure et détecter des comportements suspects.

L'objectif est de :

- générer un graphe aléatoire réaliste,
- assurer sa connexité,
- analyser ses propriétés,
- classer les sommets selon leur rôle.

2 Modèle Erdős–Rényi

Le modèle $G(n, p)$ est un graphe où chaque paire de sommets est connectée avec une probabilité p .

2.1 Choix de la probabilité

La théorie des graphes montre que :

$$p \approx \frac{\ln(n)}{n}$$

Cela garantit un graphe presque sûrement connexe lorsque n est grand.

Dans ce projet :

- pour petits graphes : $p = 0.3$
- pour grands graphes : $p = \frac{\ln(n)}{n}$

3 Génération du graphe

3.1 Algorithme

Algorithme 1 : Génération du graphe Erdős–Rényi

Entrée : n, p

Sortie : Graphe G

```
1 Créer liste d'adjacence vide;
2 pour  $i = 0$  to  $n - 1$  faire
3    $\text{adj}[i] \leftarrow \emptyset$ ;
4 pour  $i = 0$  to  $n - 2$  faire
5   pour  $j = i + 1$  to  $n - 1$  faire
6     tirer  $r \in [0, 1]$ ;
7     si  $r < p$  alors
8       ajouter  $(i, j)$ ;
9 retourner  $G$ ;
```

3.2 Code Python

```
1 import random
2
3 def generate_graph(n, p):
4     G = [[] for _ in range(n)]
5
6     for i in range(n):
7         for j in range(i+1, n):
8             if random.random() < p:
9                 G[i].append(j)
10                G[j].append(i)
11
12     return G
```

4 Correction de la connexité

Un graphe peut être non connexe. On utilise un BFS pour corriger cela.

4.1 Algorithme

Algorithme 2 : Correction de la connexité

Entrée : adj

Sortie : adj connexe

```
1 visité ← faux;
2 file ← {0};
3 tant que file non vide faire
4     extraire u;
5     pour chaque v voisin de u si v non visité marquer v;
6     ajouter v à file;
7 retourner adj;
```

4.2 Code Python

```
1 from collections import deque
2
3 def bfs(adj):
4     n = len(adj)
5     visited = [False]*n
6     q = deque([0])
7     visited[0] = True
8
9     while q:
10         u = q.popleft()
11         for v in adj[u]:
12             if not visited[v]:
13                 visited[v] = True
14                 q.append(v)
```

```
15  
16     return visited
```

5 Analyse du réseau

Une fois le graphe construit, on analyse :

- le nombre de sommets
- le nombre d'arêtes
- la densité du réseau

5.1 Densité

$$D = \frac{2m}{n(n-1)}$$

où m est le nombre d'arêtes.

6 Classification des sommets

Chaque sommet est classé selon son degré.

- faible degré → périphérique
- moyen degré → intermédiaire
- fort degré → central

6.1 Code Python

```
1 def classify(adj):  
2     deg = [len(v) for v in adj]  
3     mean = sum(deg)/len(deg)  
4  
5     classes = []  
6  
7     for d in deg:  
8         if d > mean:  
9             classes.append("central")  
10        elif d > mean/2:  
11            classes.append("intermediate")  
12        else:  
13            classes.append("peripheral")  
14  
15    return classes
```

7 Interprétation (fraude)

Ce modèle permet de simuler un réseau d'interactions.
Dans le contexte de fraude :

- les sommets centraux peuvent représenter des acteurs influents,
 - les groupes fortement connectés peuvent représenter des réseaux organisés,
 - les sommets isolés représentent des individus périphériques.
- Ainsi, la structure du graphe permet de détecter des comportements anormaux.

8 Conclusion

Ce projet montre que les graphes aléatoires permettent de modéliser des réseaux complexes.

Nous avons :

- généré un graphe $G(n, p)$,
- assuré sa connexité,
- analysé sa structure,
- classé les sommets selon leur rôle.

Ce travail peut être amélioré par des méthodes plus avancées comme la détection de communautés ou l'apprentissage automatique.